at a time in an array. For MySQL this involves mysql_result and mysql_fetch_row, respectively.

Using mysql_result is much slower than the fetch functions. PHP has to work harder to find the exact piece of data you need. First, the specified row must be referenced. Then the data in that row must be searched for a column with a matching name. You can imagine that executing mysql_result several times inside a loop can add up to a very slow script. Each call has to start at the beginning and find the appropriate data element.

Alternatively, you may fetch an entire row into an object, as we have done in most of the examples so far. This allows you to reference the exact element without searching through the entire data set. The challenge is to match up the results of the query with the array elements. If you have created a query such as

```
SELECT *
FROM user u, employer e
WHERE u.Employer = e.ID
```

you may have a hard time. You will have to examine the structure of each table to see the order of the columns. A better approach is to specify the columns you need, leaving out any you will not use. This would transform the query into something like

```
SELECT u.ID, u.Name, e.Name

FROM user u, employer e
WHERE u.Employer = e.ID
```

which specifies only three columns. You can be sure, regardless of the order of the columns in the table, that the user ID will be column zero.

Another advantage is that, since the resulting data have been narrowed to three columns, each fetch will be much smaller. The savings go all the way back to the database, because it will return only three pieces of data times the number of rows. None of the unused rows from the first version of the query will be sent through the network from the database server to PHP. In turn, PHP does not need to put them into the array.

9.9.3 When to Store Content in a Database

When we speak of content, we mean static text, perhaps containing HTML. There is no rule saying that content should never be placed in a database, or that it should always be put in a database. In the case of a bulletin board, it makes sense to put each message in a database. Messages are likely to be added continually. It is convenient to treat them as units, manipulating them by their creation dates or authors. At the